

Séance 4 : Boucle Répéter ... Jusqu'à & Structure Selon

Itérations post-test, sélecteurs scalaires, match...case et conception de menus interactifs – 3^e Sciences

1. Cadre Théorique & Conventions Officielles

⚡ RÈGLES DE CONCEPTION & CRITÈRES DE CHOIX DES STRUCTURES :

- Itérations inconnues avec **au moins une exécution obligatoire** → Structure itérative **Répéter ... Jusqu'à** (test postérieur).
- Aiguillage multiple sur un sélecteur de **type scalaire discret** (Entier ou Caractère) → Structure conditionnelle **Selon Sélecteur**.
-  **Règle** : Interdiction formelle de l'instruction `break` pour sortir prématurément d'une boucle.

A. Boucle Répéter ... Jusqu'à

```
Répéter  
  Traitements  
Jusqu'à Condition_Arrêt
```

- Évalue la condition d'arrêt **après** l'exécution du bloc (au moins 1 itération garantie).
- La boucle s'arrête dès que la condition devient **Vraie** (condition de sortie).
- Traduction Python normalisée : `while not (Condition_Arrêt):` (inversion logique de la condition).

B. Structure conditionnelle à choix multiples Selon

```
Selon Sélecteur  
  Val_1          : Traitement_1  
  Val_2, Val_3   : Traitement_2  
  Sinon          : Traitement_Par_Défaut  
Fin Selon
```

- Le sélecteur doit être impérativement de **type scalaire** (Entier ou Caractère). Les types Réel et Chaîne sont interdits.
- Traduction Python 3.10+ : instruction `match Sélecteur: case ...` (ou cascade `if ... elif ... else`).

2. Palier Débutant (Socle & Guidage)

Activité 1 — Boucle Répéter : Transmission IoT avec ACK Socle · 15 min

Mise en situation technique (Télécommunication & Réseau IoT) : Un émetteur radio IoT transmet des trames de données vers une station réceptrice. Chaque trame transmise est acquittée par un code retourné entre 1 et 6. La trame est reçue avec succès uniquement lorsque le code d'accusé de réception **ACK = 6**.

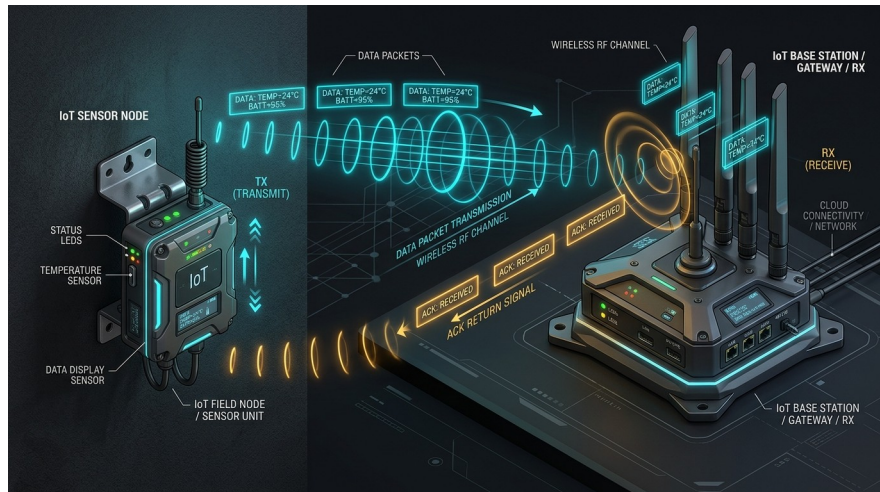


Figure 1 : Transmission des trames radio (TX) et réception du code d'acquiescement (ACK).

Algorithme de transmission radio & Questions

```
tentatives ← 0
Répéter
  ack ← alea(1, 6)
  tentatives ← tentatives + 1
  Écrire("Trame transmise... Réponse ACK : ", ack)
Jusqu'à (ack = 6)
Écrire("Connexion établie avec succès en ", tentatives, " essai(s).")
```

1. Combien de fois le bloc d'émission et d'évaluation s'exécute-t-il au minimum ? Justifier votre réponse.
2. *Exemple illustratif* : Si les tirages successifs de la fonction `alea(1, 6)` sont 3, 1, 5, 6, déterminer le nombre final de tentatives et l'affichage complet produit à l'écran.
3. Pourquoi la structure Répéter ... Jusqu'à est-elle plus adaptée et plus concise ici qu'une boucle Tant Que ?

☞ *Sur votre cahier* : Rédigez le tracé de la boucle, la trace d'exécution et écrivez la version équivalente utilisant une boucle **Tant Que** sur votre cahier.

Activité 2 — Structure Selon & Grille logistique 8×8 Socle · 15 min

Mise en situation technique (Entrepôt robotisé 4.0) : Le sol d'un entrepôt logistique automatisé est organisé sous forme d'un damier 8×8 repéré par une colonne (caractère 'a'..'h') et une ligne (entier 1..8). À l'instar d'un échiquier, les cases noires constituent des **zones de stockage** de palettes et les cases blanches sont des **voies de circulation** réservées aux robots mobiles autonomes (AMR = Autonomous Mobile Robots).

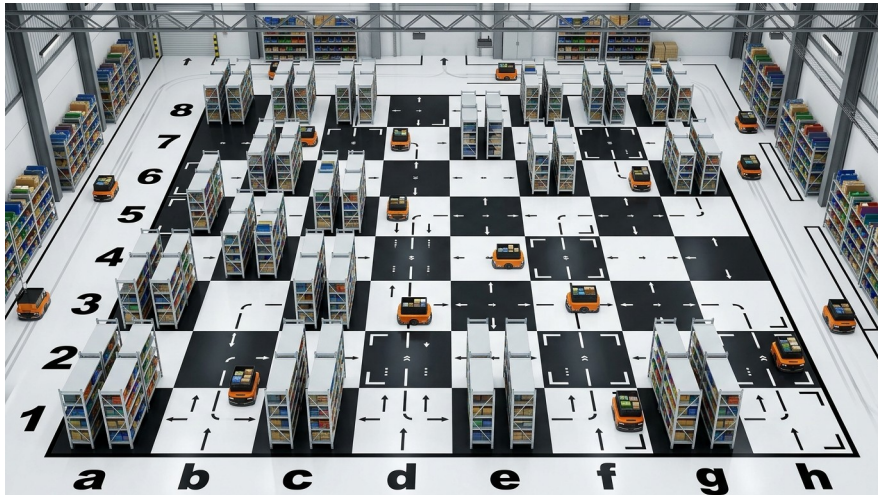


Figure 2 : Damier logistique 8×8 : cases noires (stockage) et cases blanches (circulation AMR).

Cahier des charges & TDO

1. Écrire l'algorithme d'un programme qui lit les coordonnées `col` (Caractère) et `ligne` (Entier) d'un robot et détermine s'il se trouve dans une zone de stockage ou sur une voie de circulation.
2. *Exemples* : Pour ('a', 5) → Stockage (Noire) ; Pour ('e', 6) → Circulation (Blanche) ; Pour ('z', 2) → Hors grille !
3. Compléter le Tableau de Déclaration des Objets (TDO) ci-dessous.

Objet	Type / Nature	Rôle
<code>col</code>	Caractère	Colonne repérant la position du robot ('a'..'h')
<code>ligne</code>	Entier	Ligne repérant la position du robot (1..8)
<code>col_num</code>	Entier	Numéro d'ordre numérique de la colonne (1 pour 'a' à 8 pour 'h')
<code>somme</code>	Entier	Somme <code>col_num</code> + <code>ligne</code> (détermine la couleur de la case par parité)

➤ **Sur votre cahier** : Rédigez l'algorithme complet avec la structure **Selon** `col` (convertissant 'a'..'h' en 1..8) et le test de parité sur votre cahier.

3. Palier Intermédiaire (Maîtrise & Autonomie)

Activité 3 — Traduction en Python 3.10+ avec match...case Maîtrise · 15 min

Soit un sélecteur mois $\in [1 ; 12]$. On souhaite afficher le nombre de jours correspondant au mois saisi (en considérant une année standard où février compte 28 jours) :

```
mois = int(input("Numéro du mois (1 à 12) : "))
match mois:
    case 1 | 3 | 5 | 7 | 8 | 10 | 12: print("31 jours")
    case 4 | 6 | 9 | 11:             print("30 jours")
    case 2:                         print("28 jours")
    case _:                         print("Mois invalide !")
```

1. Quel est le rôle de l'opérateur | (pipe) dans les motifs de clause case ?
2. À quelle clause algorithmique de la structure Selon correspond le motif universel case _ : ?
3. Compléter en amont par une boucle while assurant un contrôle de saisie pour forcer $1 \leq \text{mois} \leq 12$.

Activité 4 — Débogage d'une structure conditionnelle à choix multiples Maîtrise · 15 min

Un élève a écrit l'algorithme suivant pour classer des relevés thermiques réels en degrés Celsius :

```
Lire(temp) # temp est de type Réel
Selon temp
    -10.0 .. 0.0 : Écrire("Gel")
    0.1 .. 20.0  : Écrire("Frais")
    Sinon       : Écrire("Chaud")
Fin Selon
```

Code Erroné (Non conforme)	Motif du Rejet / Erreur	Correction Conforme (Normalisée)
Selon temp (type Réel)	Type Réel continu interdit comme sélecteur	Remplacer par Si ... Sinon Si ... Sinon
-10.0 .. 0.0 :	Intervalles réels non discrets	Si (temp >= -10.0) ET (temp <= 0.0) Alors

Sur votre cahier : Rédigez l'analyse technique de l'erreur et l'algorithme corrigé conforme aux normes sur votre cahier.

4. Palier Avancé (Dépassement & Défi Scientifique)

Activité 5 — Menu interactif de calculs physiques Avancé · 15 min

Mise en situation technique (Laboratoire de sciences physiques) : On souhaite concevoir une application interactive pour assister les élèves lors de travaux pratiques :

- 1 : Vitesse moyenne en m/s ($v = d / t$) avec $t > 0$.
- 2 : Énergie cinétique en Joules ($E_c = 1/2 \cdot m \cdot v^2$) avec $m > 0$ et $v \geq 0$.
- 3 : Puissance moyenne en Watts ($P = \Delta E / \Delta t$) avec $\Delta t > 0$.
- 0 : Quitter l'application.



Figure 3 : Station de physique : calculs de vitesse ($v = d/t$), d'énergie ($E_c = 1/2 m v^2$) et de puissance ($P = \Delta E/\Delta t$).

Travail demandé & TDO

1. Modéliser ce menu par une structure **Selon choix** englobée dans une boucle **Répéter ... Jusqu'à choix = 0** pour maintenir l'application active jusqu'à la demande de sortie.
2. Pour chaque option, inclure le contrôle de validité des grandeurs aux dénominateurs ($t > 0$, $m > 0$, $\Delta t > 0$).
3. Compléter le Tableau de Déclaration des Objets (TDO) ci-dessous.

Objet	Type / Nature	Rôle
choix	Entier	Option sélectionnée par l'utilisateur dans le menu (0 à 3)
d, t, v	Réel	Distance (m), durée (s) et vitesse moyenne calculée (m/s)
m, Ec	Réel	Masse (kg) et énergie cinétique calculée (J)
E, dt, P	Réel	Énergie (J), intervalle de temps (s) et puissance moyenne calculée (W)

Sur votre cahier : Rédigez l'algorithme complet avec gestion du menu et contrôles de saisie stricts sur votre cahier.

Activité 6 — Défi Scientifique : Le Lièvre et la Tortue Défi Avancé · 15 min

Modélisation probabiliste (Processus stochastique) : La course entre le Lièvre et la Tortue est simulée par un tirage répété d'un dé cubique (1 à 6). Le circuit comporte 6 cases.

À chaque tour : si le 6 sort, le Lièvre gagne immédiatement. Sinon (tirages 1 à 5), la Tortue avance d'une case et remporte la victoire dès qu'elle atteint la case 6.



Figure 4 : Simulation probabiliste : dé = 6 (Lièvre vainqueur immédiat), sinon Tortue progresse vers la case 6.

Travail demandé & Trace d'exécution

1. Écrire l'algorithme de simulation en pseudo-code à l'aide d'une boucle Répéter ... Jusqu'à (de = 6 OU case_tortue = 6).
2. Dresser le TDO des objets manipulés.

Tour	Lancer (de)	Case Tortue	Condition d'Arrêt	Événement / Décision
Tour 1	2	Case 1 / 6	Faux	Tortue progresse d'une case
Tour 2	5	Case 2 / 6	Faux	Tortue progresse d'une case
Tour 3	6	Case 2 / 6	VRAI (dé = 6)	Sortie : VICTOIRE DU LIÈVRE !

Bilan Pédagogique & Synthèse de Séance :

- **Post-condition :** La boucle Répéter ... Jusqu'à exécute son corps au moins une fois et s'arrête dès que la condition devient Vraie.
- **Traduction Python :** `while not (Condition_Arrêt):` traduit directement la boucle Répéter en inversant la condition logique.
- **Aiguillage Selon :** Exige un sélecteur scalaire discret (Entier ou Caractère). Implémenté en Python 3.10+ par `match...case`.
- **Règle :** L'instruction `break` est strictement prohibée. Toutes les sorties de boucle sont pilotées par des conditions logiques claires.

↳ *Sur votre cahier :* Implémentez la simulation en Python et testez 10 exécutions consécutives pour observer expérimentalement la fréquence de victoire de chaque concurrent.